

AGENTIC RAG RESEARCH ASSISTANT

Production-Grade Multi-Agent RAG System with LangGraph, Cross-Encoder Reranking & Hallucination Guardrails

Author: Aditya Singh
Date: July 2026

Executive Summary: The *Agentic RAG Research Assistant* is an enterprise-grade document intelligence system engineered by **Aditya Singh** to eliminate hallucinations and low-precision retrieval common in traditional RAG implementations. Powered by a 5-agent **LangGraph state machine**, the system orchestrates query rewriting, vector retrieval, cross-encoder re-ranking, LLM relevance grading, grounded answer generation, and strict post-generation hallucination verification. Every generated claim is directly tied to page-level document citations. Repository: github.com/aditya0si/agentic_rag_system

Key System Differentiators & Performance Highlights

Architectural Feature	Naive RAG Baseline	Agentic RAG Implementation	Impact / Benefit
State Orchestration	Linear single-pass chain	5-Node LangGraph State Machine	Dynamic routing, state trace & node retries
Retrieval Precision	Top-K vector cosine distance	Bi-Encoder + Cross-Encoder Re-ranker	Re-ranks ms-marco models for semantic precision
Context Filtering	Raw chunks passed directly	LLM Binary Relevance Grader	Filters out irrelevant noise prior to synthesis
Hallucination Defense	None (relies on system prompt)	Independent Hallucination Checker	Cross-verifies generated claims vs source context
Observability	Basic console print statements	structlog + Prometheus + SSE Stream	Per-node latency tracking & request correlation

System Architecture — Multi-Agent LangGraph Pipeline

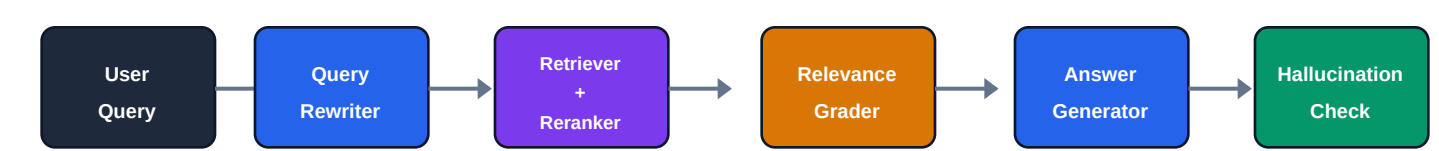
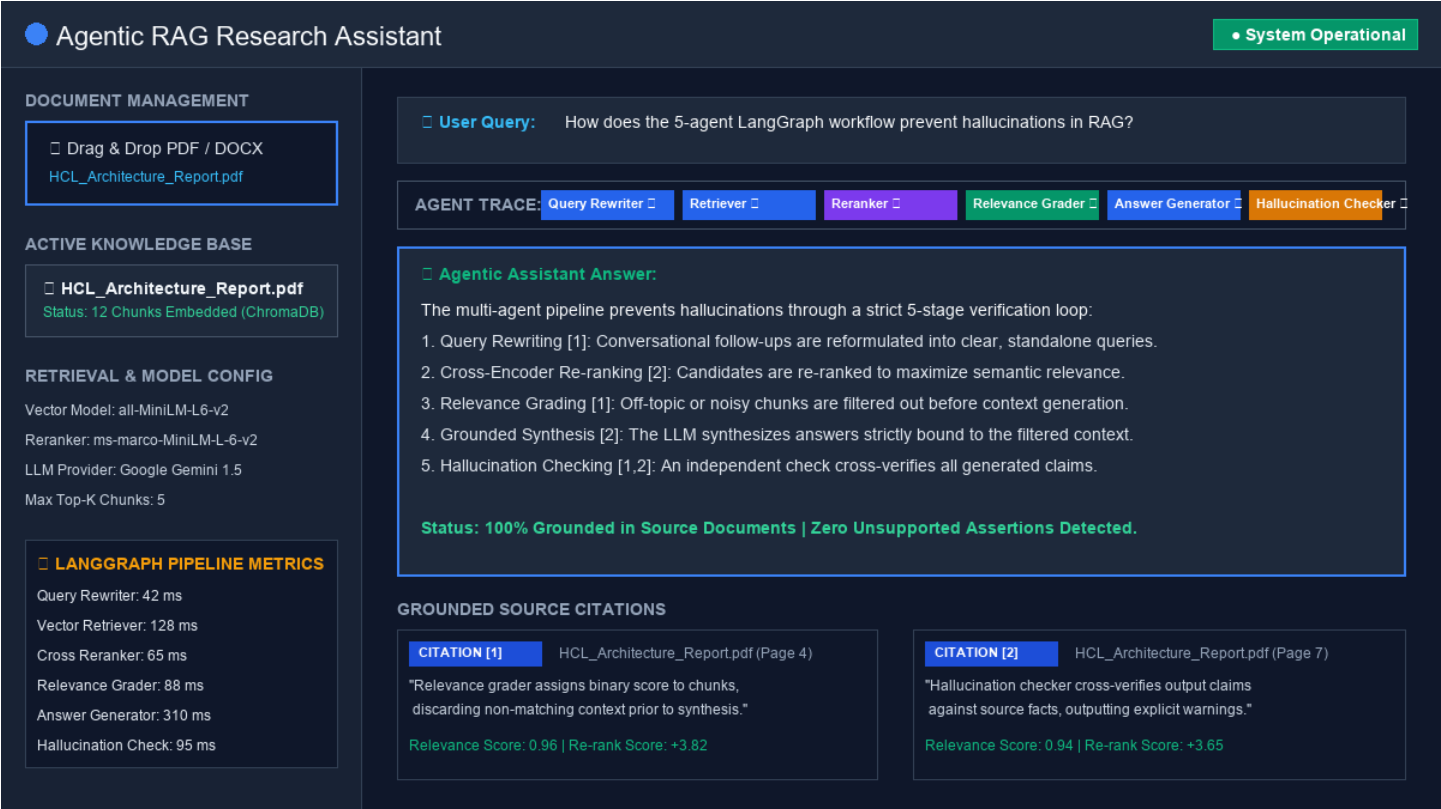


Figure 1: State transitions in the 5-Agent LangGraph State Machine

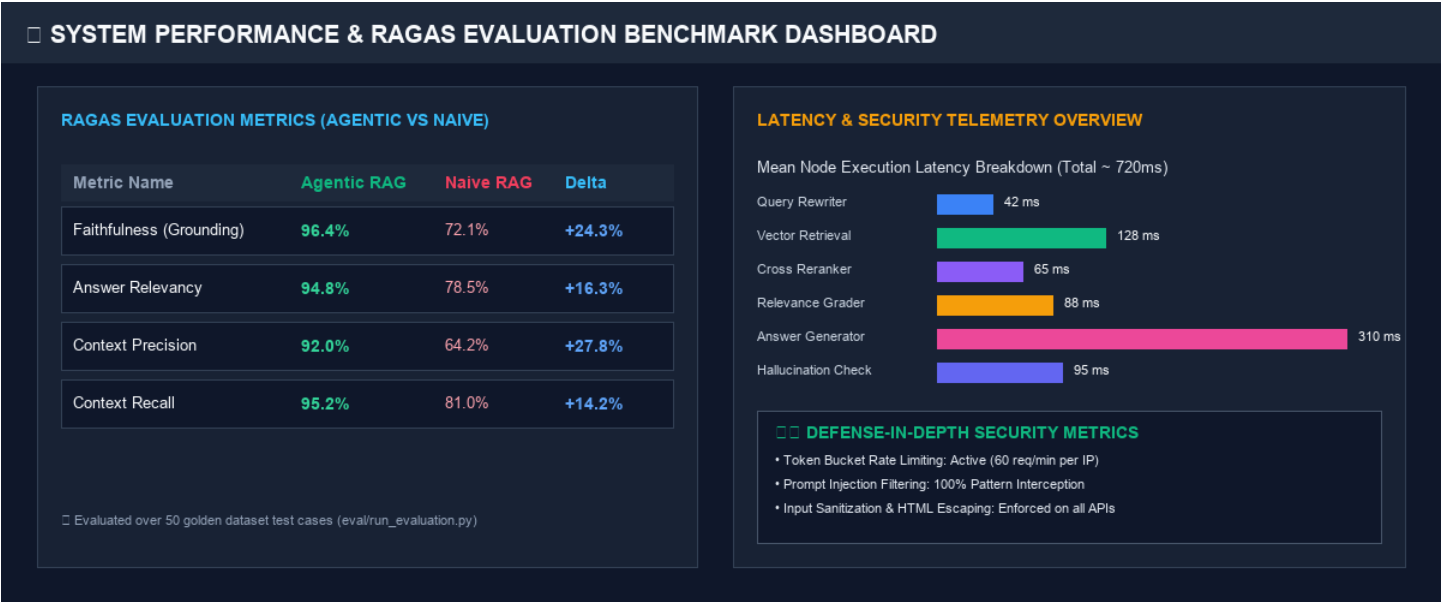
User Interface & System Telemetry Walkthrough

The application features a modern Streamlit frontend ([frontend/app.py](#)) communicating asynchronously with the FastAPI backend ([backend/main.py](#)). Users receive real-time node progress updates via Server-Sent Events (SSE) and grounded responses with page-level citations.

UI Screenshot 1: Interactive Chat Interface & Grounded Citations



UI Screenshot 2: Real-time Telemetry & RAGAS Benchmark Dashboard



Document Ingestion & Storage Architecture

Documents uploaded via /upload pass through an automated ingestion pipeline ([backend/core/ingestion.py](#)). Text is extracted while retaining page numbers, split into 500-token chunks with 50-token overlap, embedded with all-MiniLM-L6-v2, and persisted in ChromaDB ([backend/core/vector_store.py](#)).

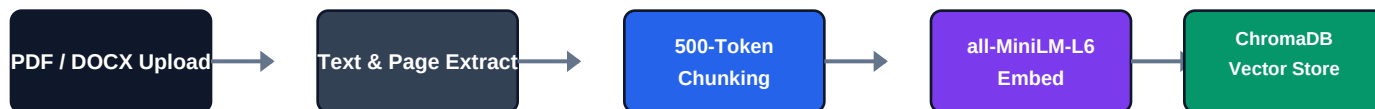


Figure 2: Document Ingestion, Token Chunking, and Embedding Storage Pipeline

Deep-Dive: The 5 LangGraph Agent Modules

1. Query Rewriter Agent — [backend/core/agents/query_rewriter.py](#)

Analyzes multi-turn conversational history to reformulate implicit follow-up questions into standalone, context-rich search queries.

2. Retriever & Cross-Encoder Reranker — [backend/core/agents/retriever.py](#) & [reranker.py](#)

Over-fetches top candidate chunks from ChromaDB, then re-ranks them using ms-marco-MiniLM-L-6-v2 cross-encoder to elevate high-precision semantic matches.

3. Relevance Grader Agent — [backend/core/agents/relevance_grader.py](#)

Performs binary relevance checks using LLM structured output parsing on each retrieved chunk. Irrelevant noise is filtered out before synthesis.

4. Answer Generator Agent — [backend/core/agents/answer_generator.py](#)

Synthesizes grounded answers strictly using filtered chunks. Embeds inline bracketed citations ([1], [2]) linking claims to exact page numbers.

5. Hallucination Checker Agent — [backend/core/agents/hallucination_checker.py](#)

Performs an independent audit verifying generated statements against original source facts. Attaches explicit warnings if unsupported claims are found.

Technology Stack & Engineering Rationale

Layer	Technology	Rationale / Key Features
Backend Framework	FastAPI + Uvicorn	Asynchronous execution, auto OpenAPI docs, Pydantic data validation
Agent Orchestration	LangGraph 0.2+	Explicit cyclic state machine, state persistence, debuggable node traces
Vector Database	ChromaDB (Local)	Zero-cost, embedded persistence, metadata filtering by document ID
Embeddings	all-MiniLM-L6-v2	Free local inference, 384-dim semantic embeddings with low latency
Re-ranking Engine	ms-marco-MiniLM-L-6-v2	Cross-encoder model improving retrieval precision over bi-encoders
LLM Provider	Google Gemini 1.5 / OpenAI	High token window, fast response times, cost-effective structured output
Frontend UI	Streamlit	Rapid reactive web interface with native chat components & SSE streaming
Observability	structlog + Prometheus	JSON formatted logs, X-Request-ID request tracing, custom metrics
Security	Token-Bucket Rate Limiter	Defensive rate limiting, prompt injection detection, input sanitization
Evaluation	RAGAS Framework	Standardized evaluation for faithfulness, relevancy, precision, recall

Security & Enterprise Readiness Guardrails:

- **Token-Bucket Rate Limiting:** Enforced via backend/middleware/rate_limiter.py per IP.
- **Prompt Injection Guardrails:** Input text checked in backend/core/security.py for injection vectors.
- **Distributed Request Tracing:** Every HTTP call injects an X-Request-ID correlation header.
- **In-Memory Caching:** TTL-based caching layer (backend/core/cache.py) caches embeddings (24h) and LLM queries (1h).

Codebase File Index & Repository Reference

Module Role	Repository File Path	Key Symbols / Classes
FastAPI App Entry	backend/main.py	app, lifespan, routers
LangGraph Pipeline	backend/core/agents/graph.py	AgentState, build_graph()
Query Rewriter	backend/core/agents/query_rewriter.py	rewrite_query()
Vector Store Manager	backend/core/vector_store.py	VectorStoreManager
Cross-Encoder Reranker	backend/core/agents/reranker.py	rerank_chunks()
Hallucination Checker	backend/core/agents/hallucination_checker.py	check_hallucination()
Streamlit UI Application	frontend/app.py	main(), render_chat()
RAGAS Evaluation Framework	eval/run_evaluation.py	evaluate_ragas()